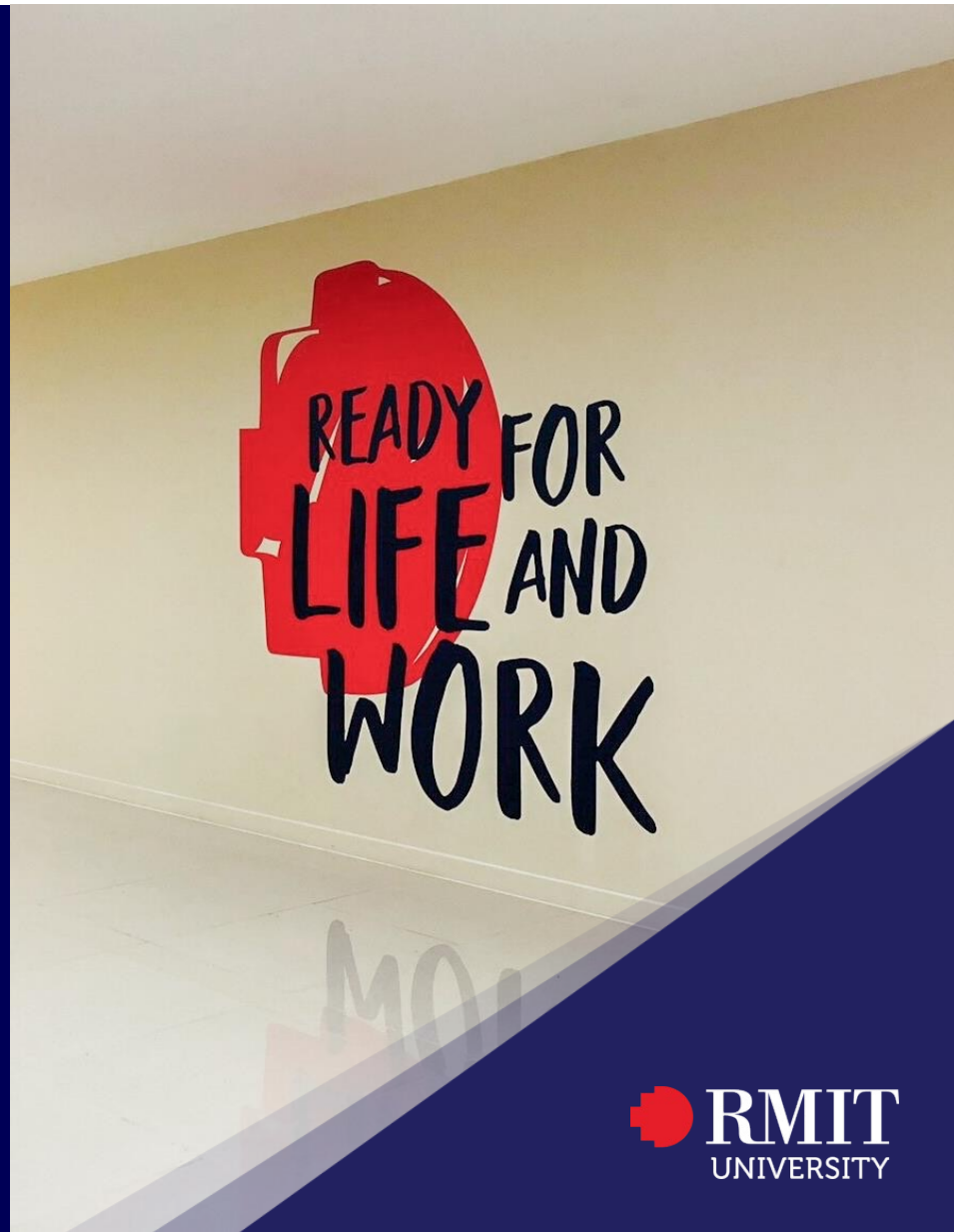


Practical Matters in Machine Learning

COSC2753 Machine Learning
Semester 2, 2026



Agenda



- Feature selection
- Feature construction
- Ensemble methods

Feature Selection

Feature Selection



Machine Learning algorithms, in practice, perform better when using a subset of only the most relevant features

- Intuitively, the learner makes fewer considerations (**curse of dimensionality**)
- **Computational theory** actually gives us this!
 - Fewer features reduces the dimension of the hypothesis space
- Obviously, this presumes the subset of features is still sufficient for a low true error

Domain Expert



Expert knowledge of the domain (or learning problem) can be very useful

- The expert can identify aspects of the domain, training data, learner to simplify the complexity learning
- That is, the expert employs **heuristics** to improve the accuracy and speed of learning
- As with all heuristic methods, be careful:
 - The expert may be wrong
 - Complex learners often identify patterns that are not obvious to an expert

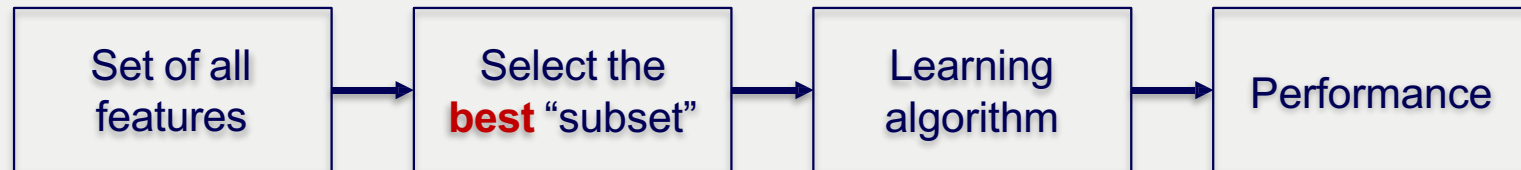
Subset Selection



Subset Selection is a basic form of feature selection that can be used with any learner

- The process of choosing a subset of the features
- A form of hill-climbing search - the subset is chosen so long as the training error improves
- Three categories of methods for subset selection:
 - 1- **filter methods**,
 - 2- **wrapper methods**, and
 - 3- **embedded methods**

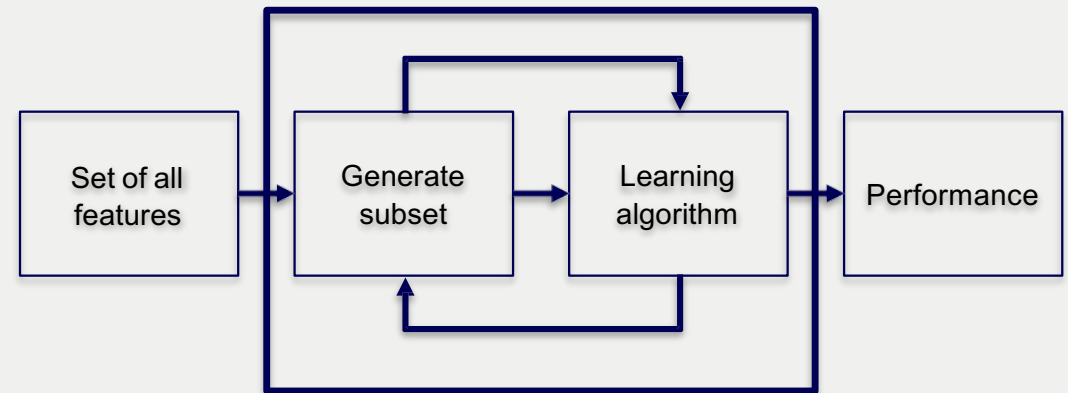
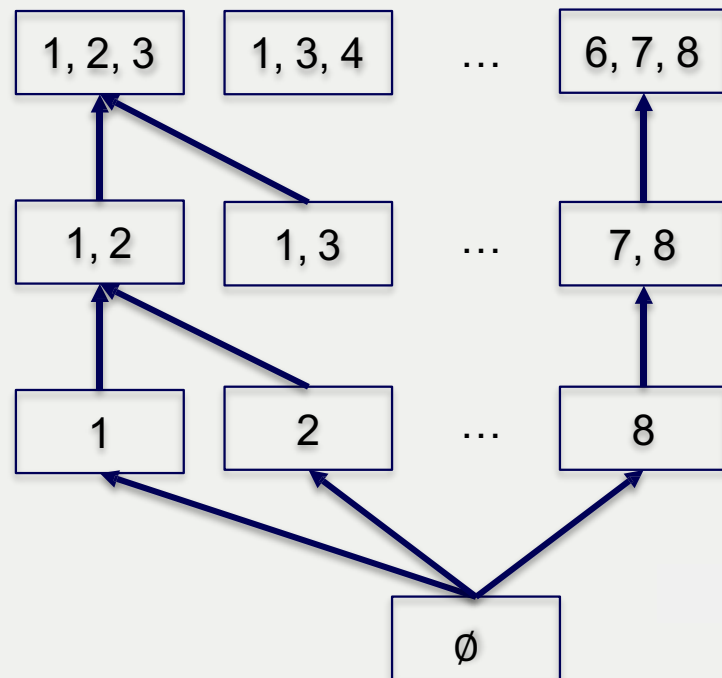
Filter Methods



What is "**best**"?

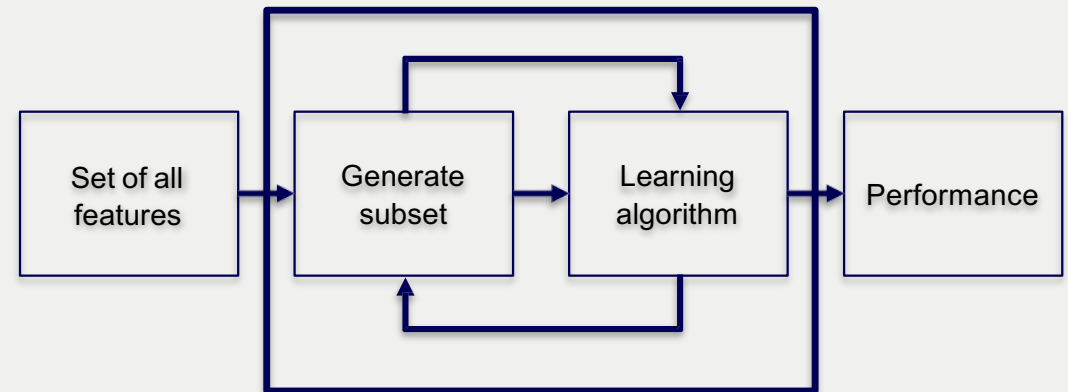
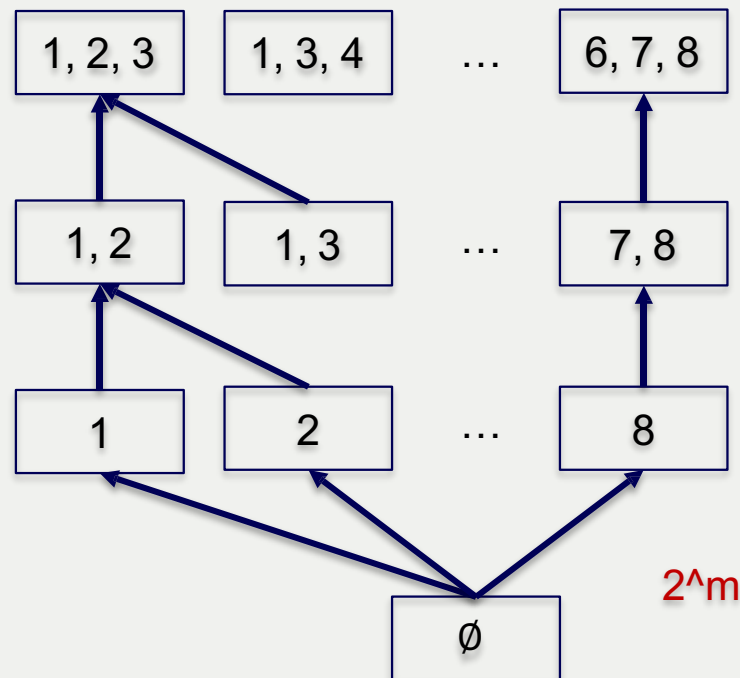
- Domain expertise
- Less correlation
- ... (problem depending)

Wrapper Methods



What are all the possible subsets?

Wrapper Methods



What are all the possible subsets?

Wrapper Methods–Search Methods



Forward selection:

- Incrementally add new feature(s) and retrain the learner
- Continue adding features so long as the training error improves (or is similar)
- Useful for finding a small number of important features

Backward selection:

- Start with all features and incrementally remove feature(s)
- Useful for removing a small number of unimportant features

Embedded Methods



Perform feature selection within the learning algorithm itself

Feature selection that is **embedded** in the model training process

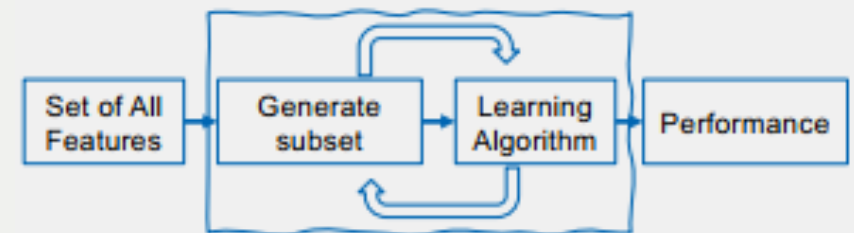
These methods include regularisation (e.g., Ridge, Lasso) and tree-based feature importance

Feature Selection



Factors to consider when deciding feature selection:

- Evaluation:
 - How to do Hyper parameter selection?
 - Can there be Overfitting?
- Complexity of the model - intuition.
- Training speed.



Feature Construction

Feature Construction



Feature Construction is the process of **inventing additional features** to add to a learning problem

The constructed features don't contain more information than the original features

However, they may represent the information in a form that is easier for a learner to use

Once again **expert knowledge** of the domain (or learning problem) can be very useful

An expert can determine what type of features may be necessary to construct

Feature Construction



<https://mc.ai/deep-learning-convolutional-neural-networks-2/>

Basic Feature Construction



In principle, many pre-processing techniques can be considered to be feature construction

“Cleaning” the data set

Separating out different forms of data

Basic Feature Construction



Other basic forms of feature construction include:

Discretisation, that partitions continuous domains into discrete (sequential) categories

Binarisation, partition data into two (binary) categories

One-hot encoding, label encoding, ...

Normalisation, scaling the relative domains of each feature to a consistent range, while still preserving the relative distance between value along each individual domain



Projections



Projections:

Reduces dimensionality

- Apply a transformation to collapse multiple features/attributes
- That is, project a high-dimension state space onto a lower (or single) dimension state space

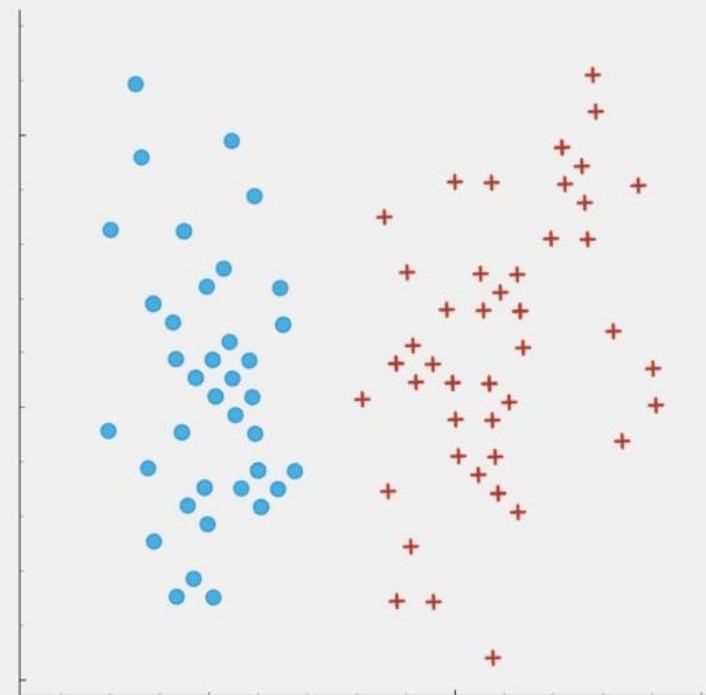
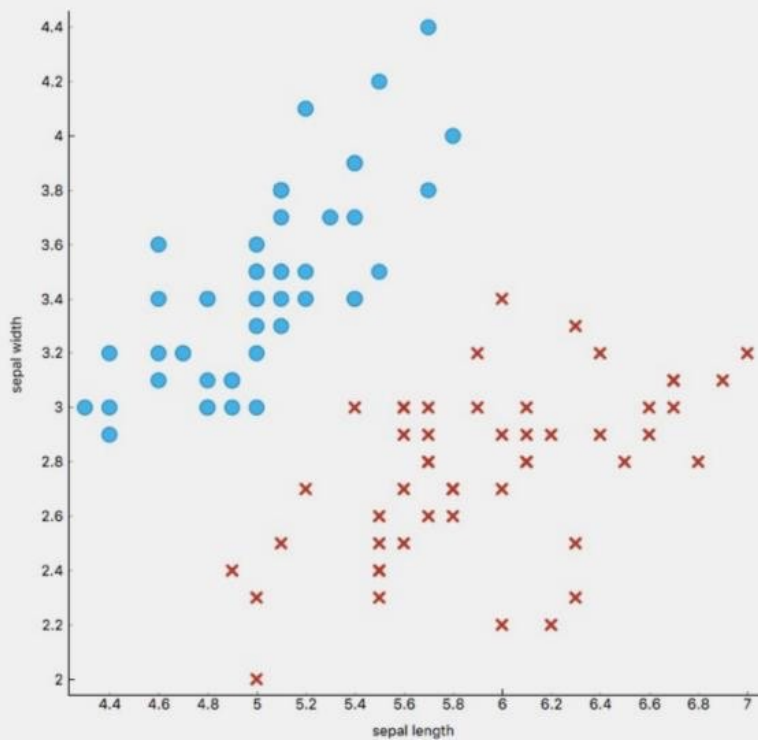
$$z = \mathbf{T}\mathbf{x}$$

Care must be taken to preserve useful properties of the features such as:

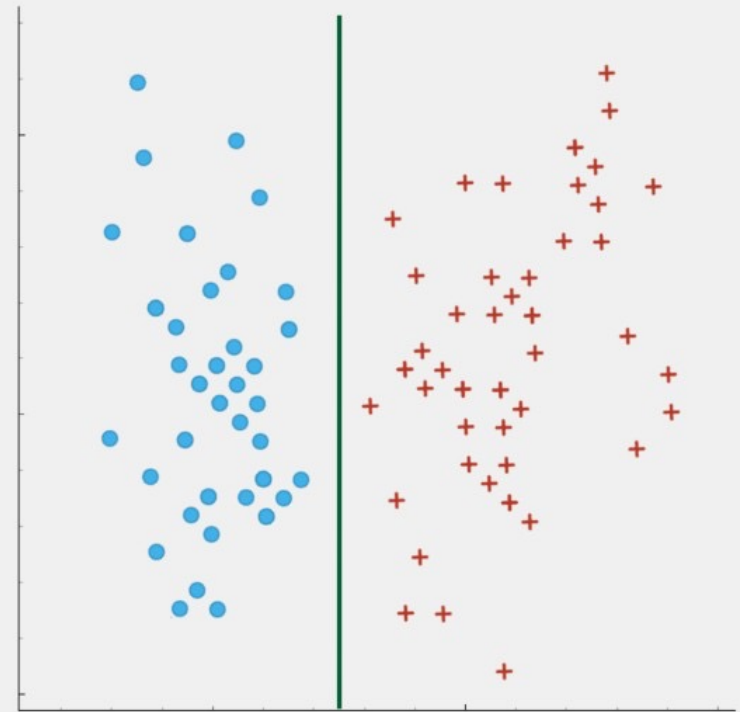
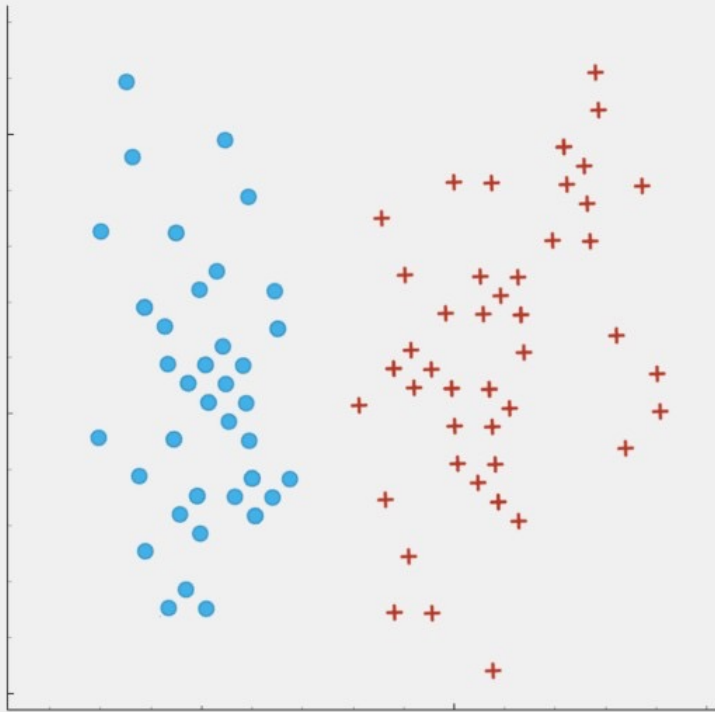
- maintain relative distance between points (preserves relative scale)

For example: Gaussian Random Projection

Projections



Projections



Kernels



Kernel Functions:

Similar to projections (depends on the particular sub-field of ML literature)

More generally, some form of function:

$$K(x)$$

Common kernels, include **polynomial**, **radial basis**, and **sigmoid**

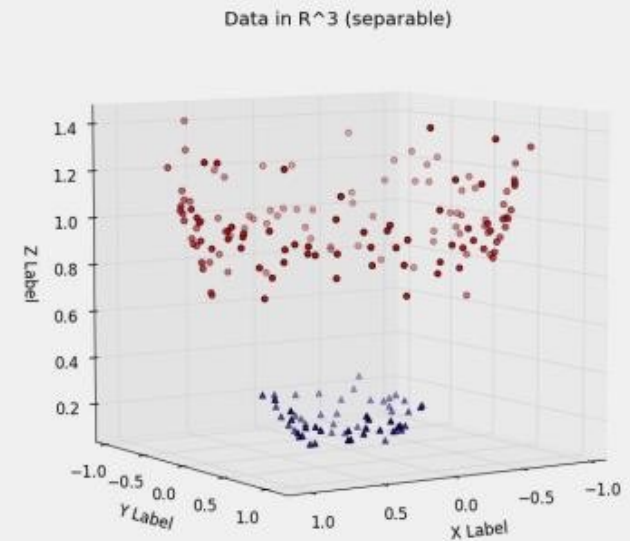
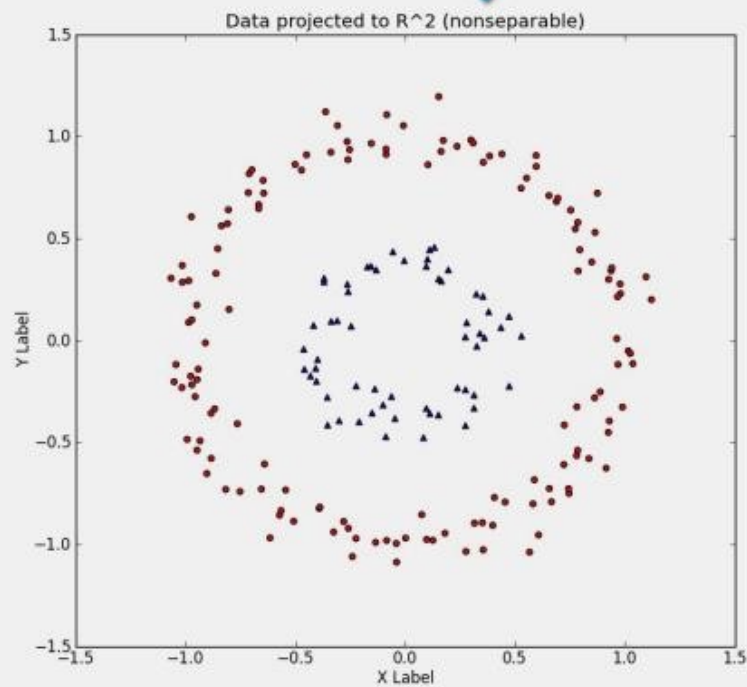
Kernel functions are generally used with specific ML algorithms:

In particular with Support Vector Machines (SVM) which we won't cover in this course

Kernels



$$z = K(x) = x_1^2 + x_2^2$$



Principal Component Analysis

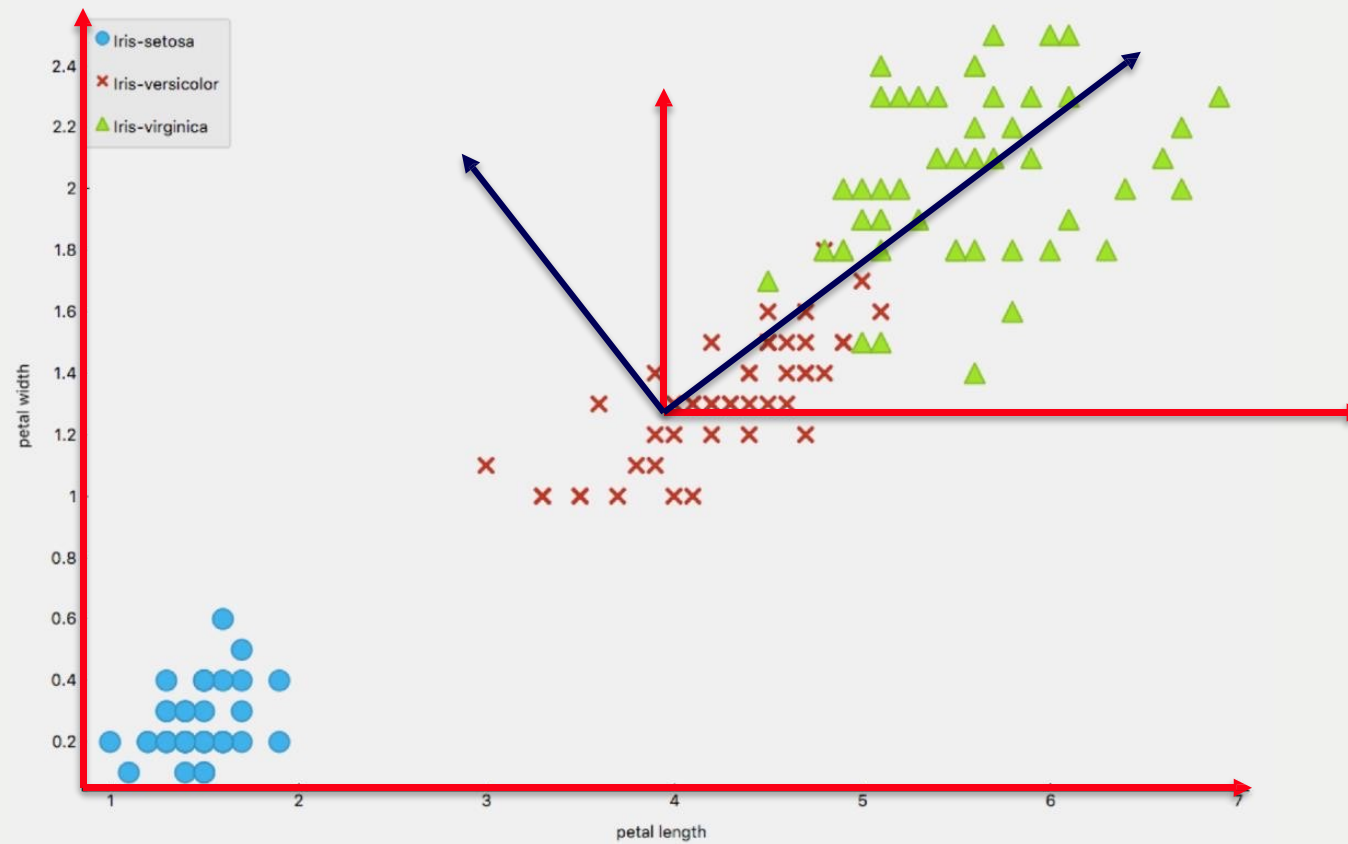


Principal Component Analysis (PCA) is a popular projection method for feature construction

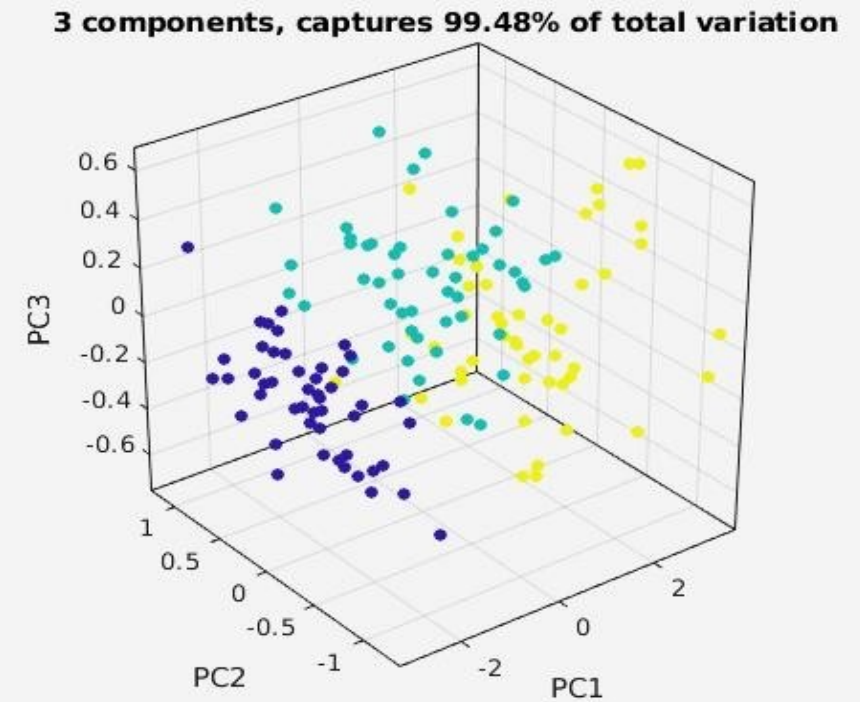
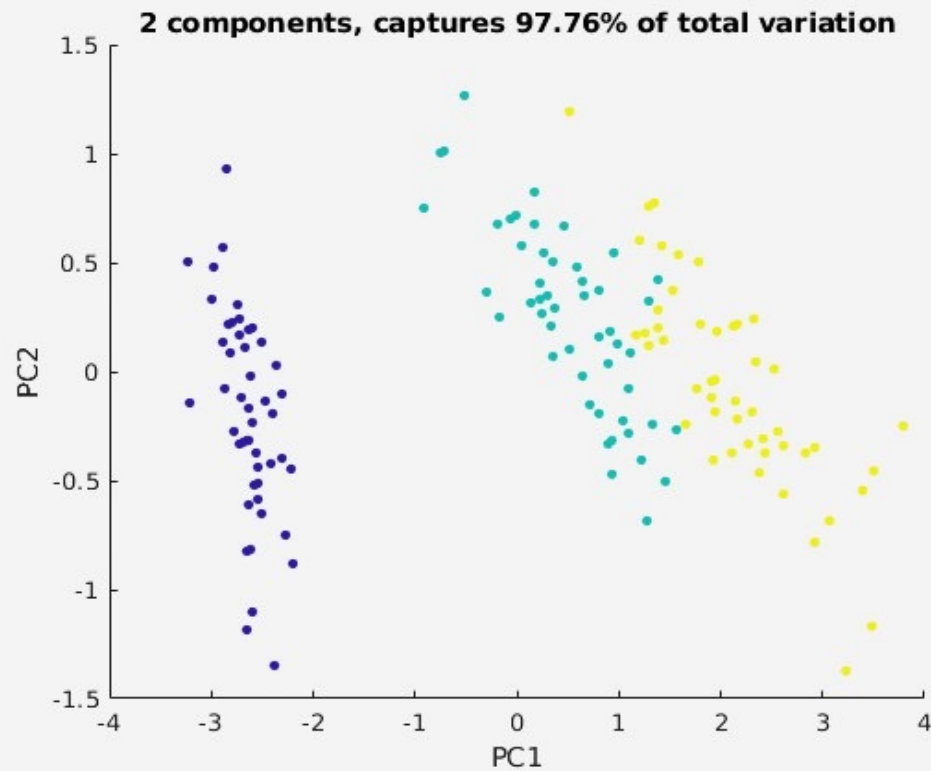
The projection reduces the state space of a set of training data into its “principal components”

The principal components are orthogonal vectors that define a projected state-space, so that the components are uncorrelated

PCA – Illustrative Example



PCA – Illustrative Example



PCA – 3D Space Example

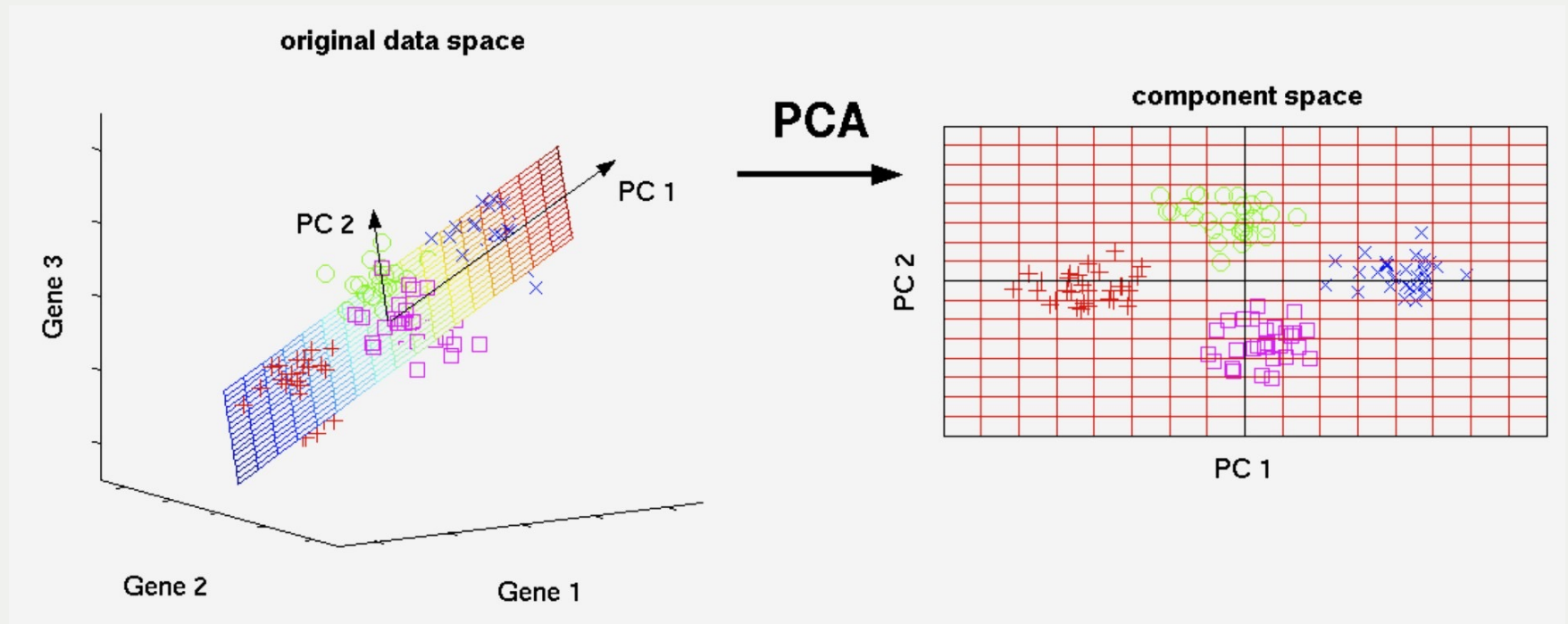


Image source: nlpca

Ensemble Methods

General Problem with “Single-model”



The supervised learning methods that have been covered so far all build a **single-model**

The model forms a single hypothesis for over the entire learning problem and state space

General Problem with “Single-model”



“Weak” learners that fail to find a good model

Learners are susceptible to bias in the training data

- k-fold cross-validation can measure the “average error” of a model, but
- A model to use must still be chosen, that is, which training-data should be used?

Learners may struggle if the hypothesis is “too complex” (overfitting versus model complexity)

Ensemble Methods



Rather than learn a single “catch-all” model:

Learn a series of smaller **expert** models:

- Each model may be an “expert” of:
 - A particular sub-set of the training data, or
 - A sub-set of the learning problem

The **meta model** votes over the experts

Ensemble Methods



The idea is that:

- Each model in the ensemble is a domain expert at one thing
- An individual expert model is a bad hypothesis for the whole learning problem
- But, the meta model is good at identifying the correct expert model(s)
- Thus the overall performance of the ensemble is better than a single model
- However, the final model may be significantly harder to interpret

Why does it work?



Ensemble methods work because:

It is often “easier” to learn smaller models

- Less training data is required
- Less sophisticated algorithms are required
- Learning is faster

Sub-divide learning a complex hypothesis into:

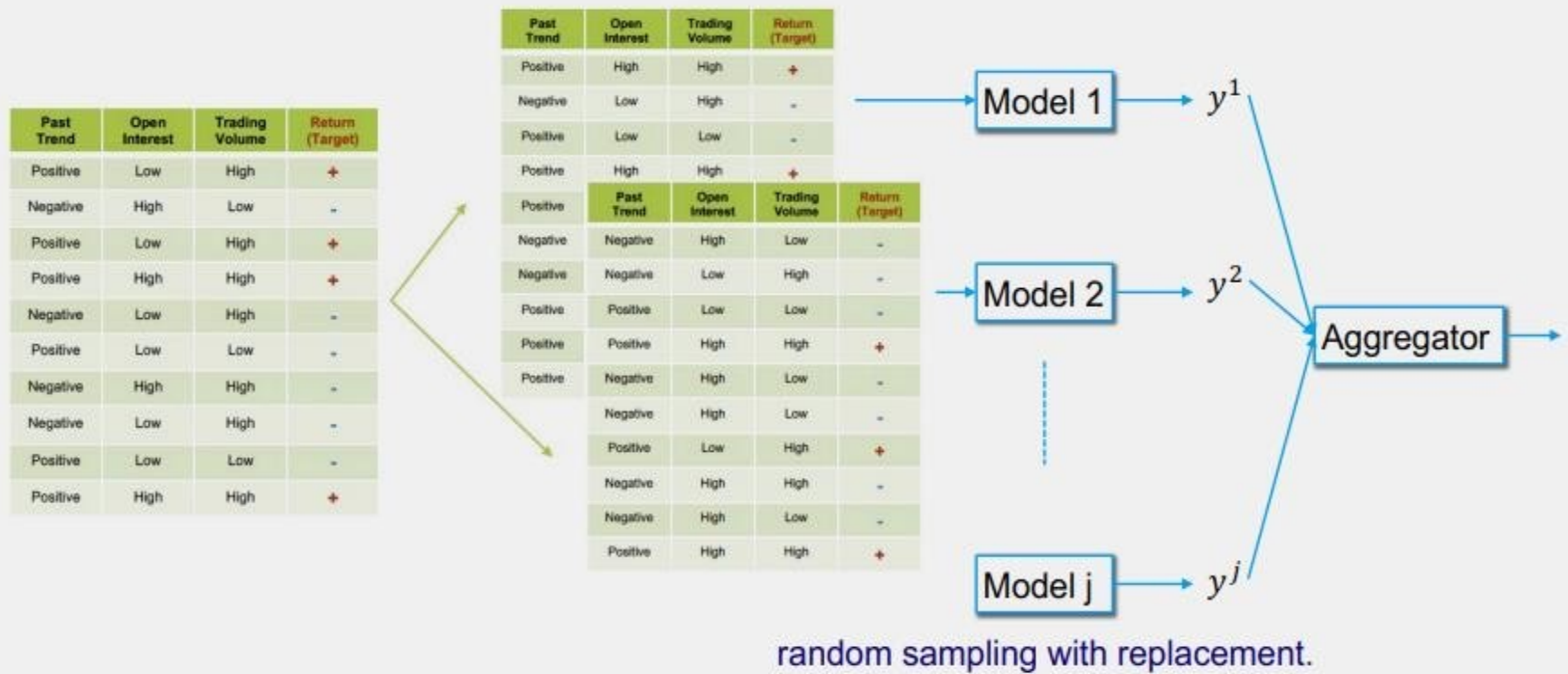
- A hypothesis that uses suitable attributes for the actual prediction/classification/etc.
- A meta-hypothesis that uses suitable attributes to identify how the prediction should function

How to create an ensemble of models?



Bagging
Feature bagging
Boosting

Bagging



Boosting



Boosting:

Improves the performance of **weak learners**, that is, learners which are not much better than random

Rather than aggregate over random learners, use the **mistakes of previous expert models** when training subsequent experts

- That is, each subsequent model is trained to be an expert on the training examples that are incorrectly classified by previous models
- This “boosts” the performance of the overall ensemble
- That is, deal with the “easy” examples first, then progressively work with the “harder” examples

The intuition behind this idea is that the **expert models should complement each other, rather than be completely independent**



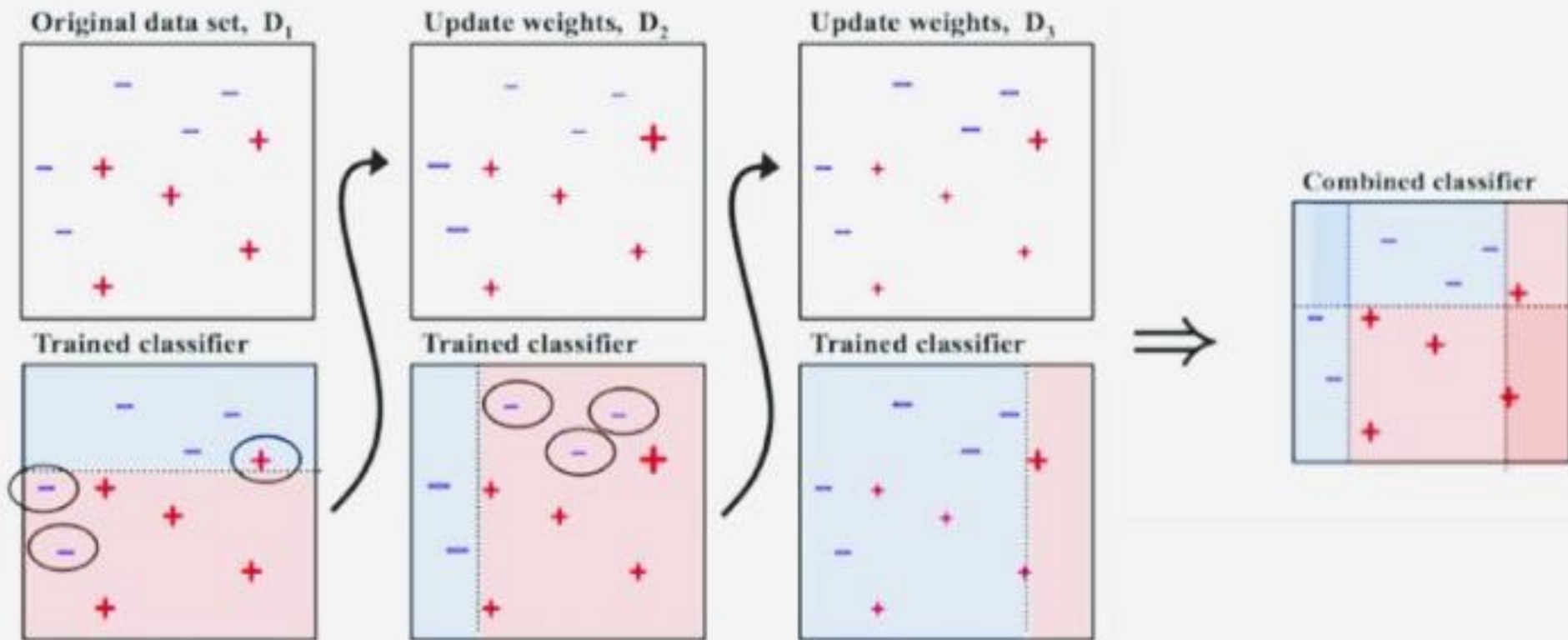
AdaBoost



One of the more common boosting algorithms. The general idea is:

- Use a weighted training data set, that is, each training example is modified by a weight
- Weight each example proportionally to the error (performance) of the expert models for that example
- Generate a variable number of expert models
- Adjust the “worth” or effectiveness of each expert model by their overall error
- Classify test data according to the worth of each expert

AdaBoost



AdaBoost



Training:

Assign an equal weight ($a_i = \frac{1}{N}$) to each training example in S

For up to t iterations:

 Learn $f_t(x)$ using weighted data set S and get weighted error e

 Compute coefficient $w_t = \frac{1}{2} \log \frac{1-e}{e}$

 Update weight $a_i = a_i e^{-w_t}$ if $f_t(x_i) = y_i$ or $a_i = a_i e^{w_t}$ otherwise

 Normalize $a_i = \frac{a_i}{\sum a_i}$

Prediction:

$$\hat{y} = \text{sign}(\sum_t w_t f_t(x))$$

AdaBoost versus Random Forest



Random Forest

1. Generate a set of random trees
 - Boot-strapped data sets
 - Feature bagging
2. Can be full size tree
3. Each tree has an equal say in the final decision

AdaBoost

1. Mostly weak learners (stumps)
2. Generate a sequence of trees where each subsequent model considers the mistakes made by previous trees (**boosting**)
3. Each tree has a say proportional to how “good” that tree is

Summary



Feature Selection

- Domain Expert
- Subset Selection

Ensemble Learning

- Bagging
- Feature Bagging
- Boosting
- AdaBoost

Feature Construction

- Projections
- Kernels
- Principal Component Analysis

Thank you!